Distributed Caching: The Secret to High-Performance Applications

BYTEBYTEGO
NOV 21, 2024 • PAID

The demand for high-speed, high-performance applications has skyrocketed in rec years.

With users expecting real-time responses, especially in sectors like e-commerce, finance, gaming, and social media, even a few milliseconds of delay can lead to a po user experience, potentially impacting customer satisfaction and revenue.

One core technique to accelerate data retrieval and improve application responsiveness is caching.

Caching works by temporarily storing frequently accessed data in a high-speed stor layer, often in memory. This allows applications to retrieve information faster than they had to pull it from the primary database each time. A single cache node is ofte sufficient for smaller systems or applications with a limited user base to store and serve frequently requested data.

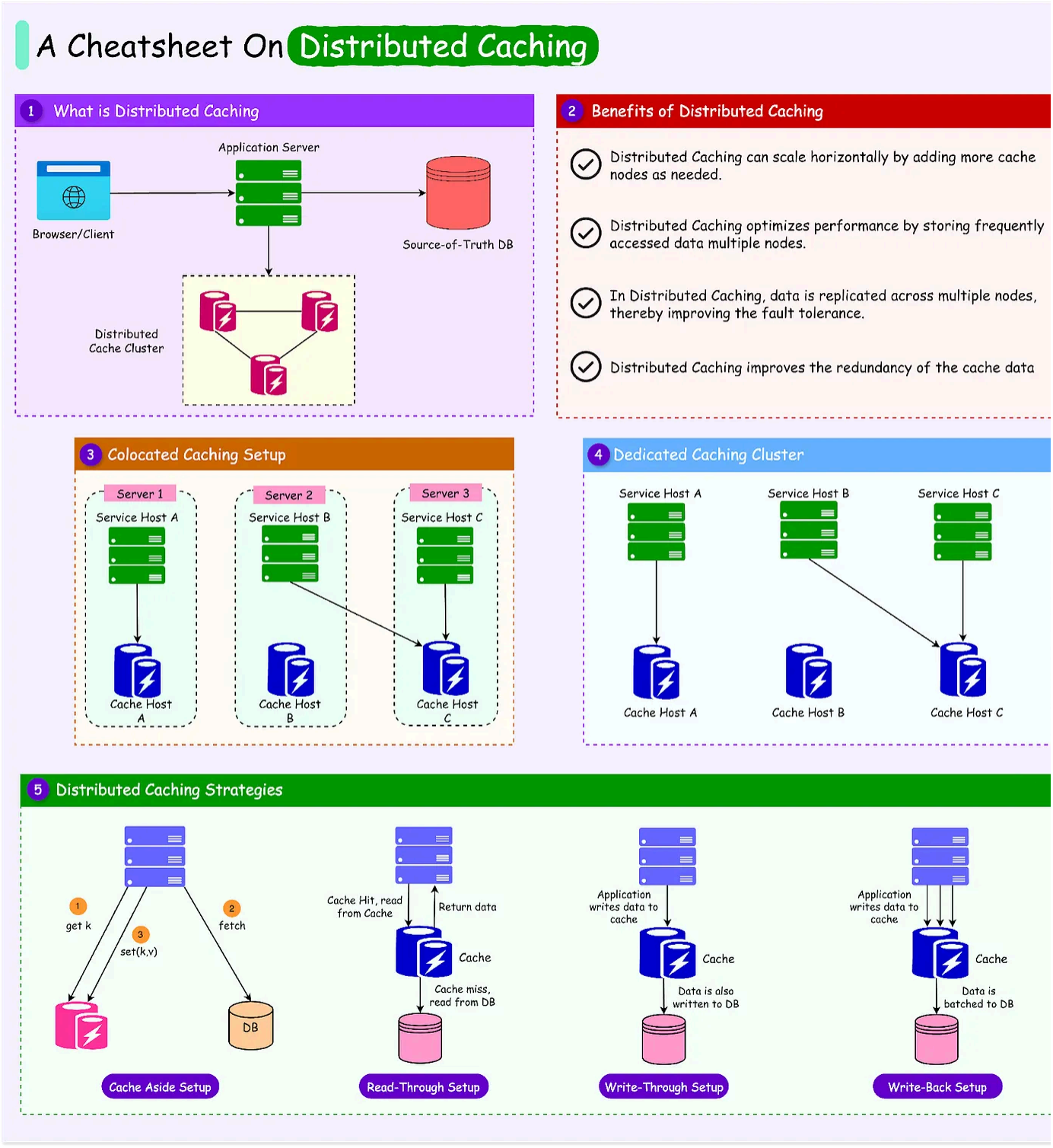
However, as systems grow, this setup faces limitations. Relying on a single-node ca to serve large-scale, high-traffic applications can lead to multiple problems.

This is where distributed caching comes into play.

Distributed caching involves spreading the cached data across multiple servers or nodes, allowing the cache to scale horizontally to handle large-scale applications. W a distributed cache, data is stored across multiple locations, meaning a single-node

failure doesn't compromise the entire cache, and the system can continue to serve requests seamlessly.

In this article, we'll explore the concept of distributed caching in depth. We'll look how it works, discuss its key components, and examine common challenges and bes practices for implementation.



Why Distributed Caching?

As mentioned earlier, traditional single-node caching works well for applications w moderate user bases and limited data storage needs, but as applications grow, this

approach can quickly run into major limitations.

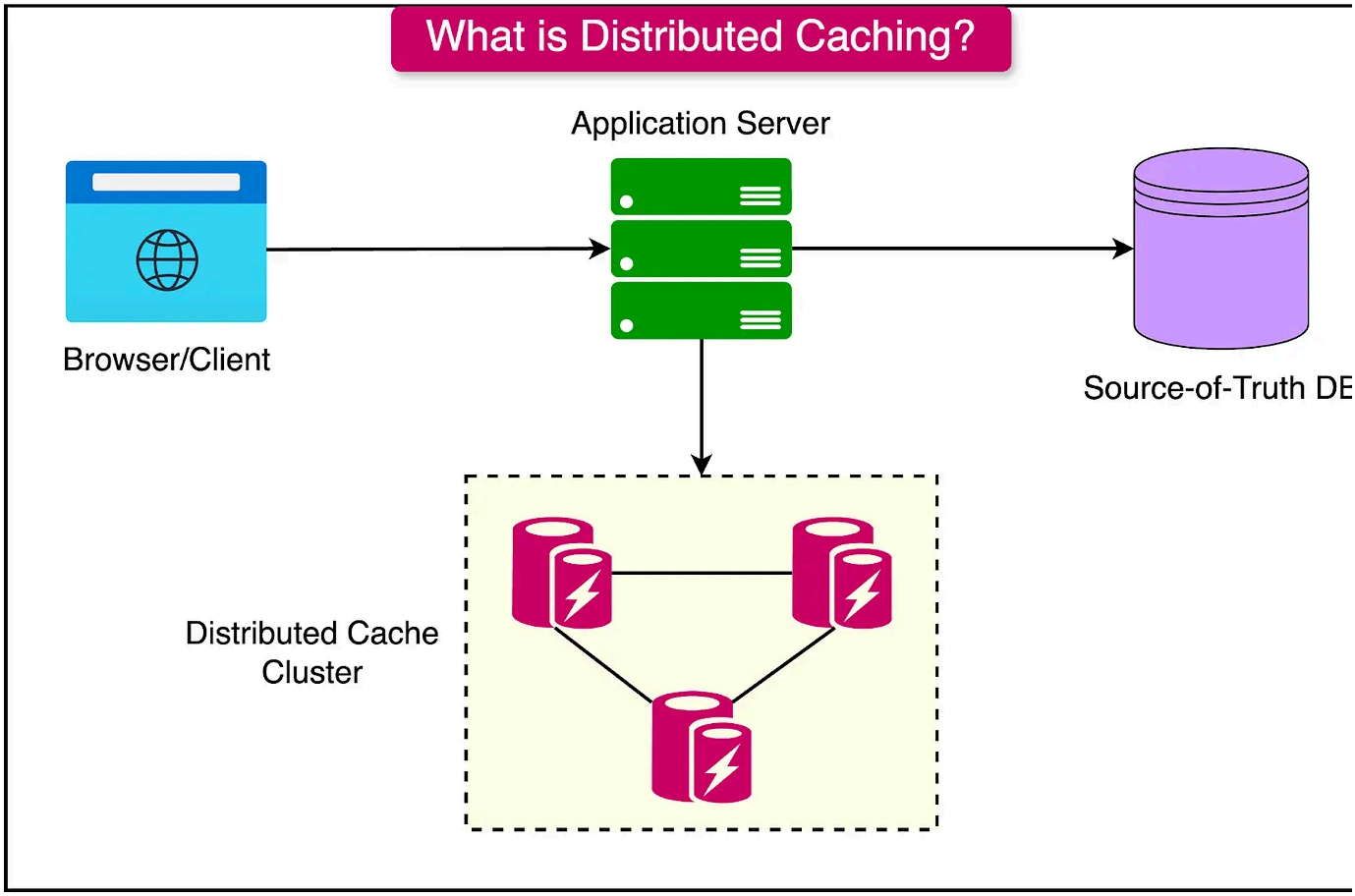
Some significant limitations of single-node caching are as follows:

- **Scalability Constraints:** Single-node caches are limited by the memory and processing power of a single server. As data volume and user requests grow, thi single cache cannot keep up with the demand, leading to slow response times a reduced performance.
- **Single Point of Failure:** Relying on a single cache server creates a vulnerability this server fails, the entire cache becomes unavailable, forcing the application t fetch all data from the primary database. This can cause critical delays, especia under high load conditions.
- **Inefficient Load Management:** A single-node cache may struggle to handle spi in traffic, such as during peak times for e-commerce sites or live event streami This overload can lead to dropped requests or severely reduced performance.
- **Limited Redundancy:** With data stored on one node, there's no backup if the n becomes unavailable. This lack of redundancy is a big weakness in maintaining data availability.

Benefits of Distributed Caching

Distributed caching addresses these limitations by spreading cached data across multiple nodes, creating a resilient, scalable caching layer that can grow with an application's demands.

The diagram below shows a typical distributed caching setup.



Here's how distributed caching enhances scalability, performance, and fault toleran

- **Scalability:** Distributed caching can scale horizontally by adding more cache nodes as needed. Each node holds a portion of the data, reducing the load on ai single node and allowing the system to handle millions of users and vast datase seamlessly.
- **Performance Optimization:** By storing frequently accessed data across multipl nodes, distributed caching improves response times.
- **Fault Tolerance:** In a distributed cache, data is replicated across multiple node so if one node fails, other nodes can continue to serve cached data without interruption. This redundancy is crucial for high-availability applications since ensures that the system remains operational even if individual nodes go offline

Practical Examples

Here are some practical examples where distributed caching becomes necessary:

- **Peak-Time E-Commerce Traffic:** E-commerce sites experience high traffic dur peak events, such as Black Friday or holiday sales. Distributed caching helps distribute the load across multiple nodes, preventing system slowdowns and providing users with a fast, reliable shopping experience. Key data like product information, prices, and user sessions can be cached across several nodes, allowing quick access even under heavy load.
- **Live Sports Streaming:** For live sports platforms, delivering real-time video fee and game statistics to millions of viewers is essential. Distributed caching allov video fragments and metadata to be cached across multiple nodes, ensuring smooth streaming and minimizing latency.

How Distributed Caching Works?

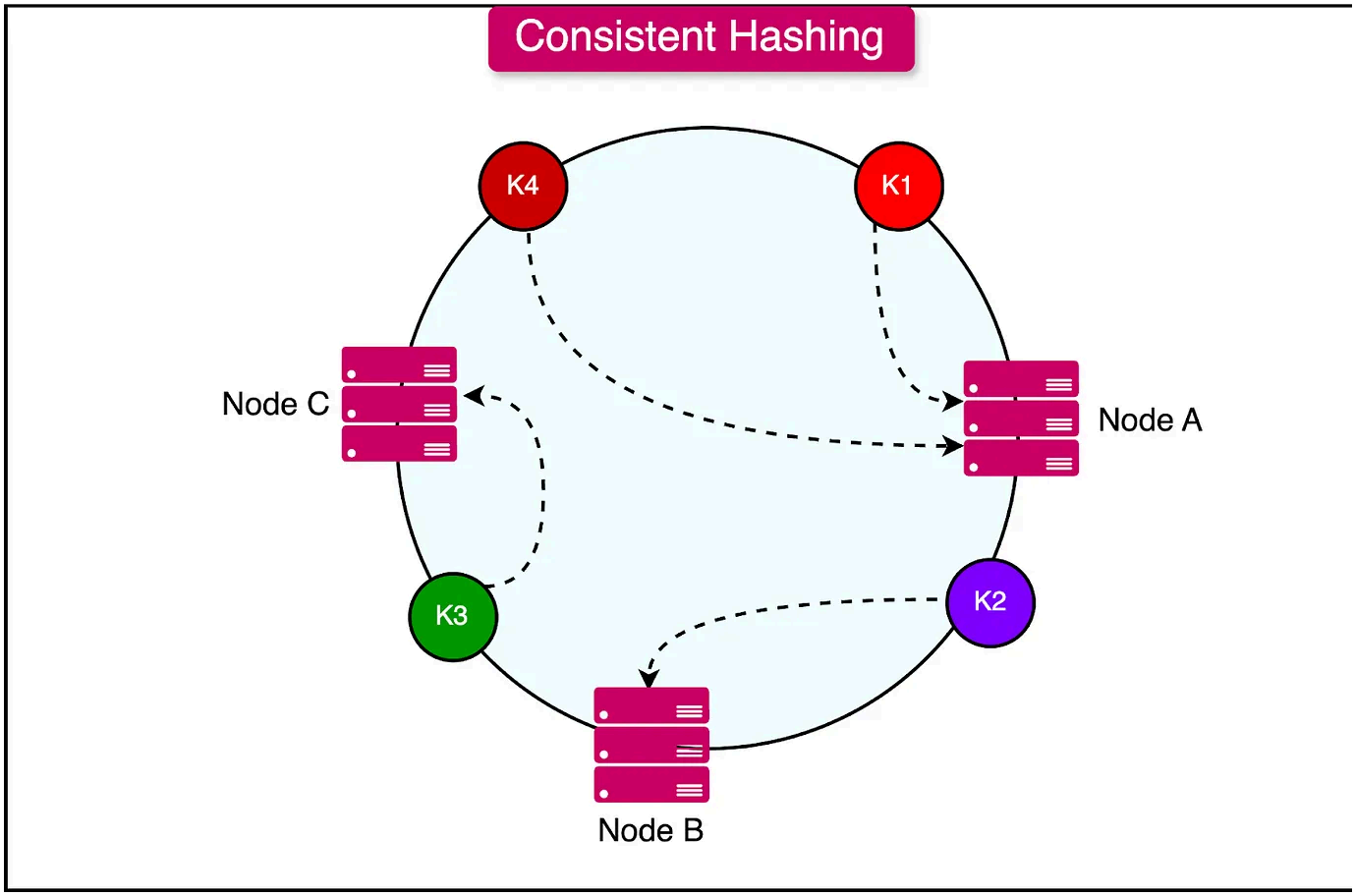
The architecture of distributed caching is designed to address the scalability and performance needs of large-scale applications by distributing cached data across multiple servers or nodes.

Here are the key components of a typical distributed caching architecture:

- **Cache Nodes:** In a distributed cache, data is spread across multiple servers or nodes. Each cache node holds a portion of the cached data, allowing the entire cache to scale horizontally as more nodes are added. The cache nodes work together to respond to data requests, share the workload, and provide redundar in case of node failure.
- **Client Libraries:** A client library acts as an intermediary between the applicati and the distributed cache nodes. It determines which node stores the data the application needs and routes requests accordingly. The client library also hand data distribution, manages data retrieval, and ensures the application interacts with the correct cache nodes.

- **Consistent Hashing:** Distributed caches often use consistent hashing to distrib data efficiently across multiple nodes. This hashing technique assigns data to specific nodes based on a hash of the data key. Consistent hashing is designed minimize data movement when nodes are added or removed, helping maintain cache stability and reducing reallocation costs.
- **Data Replication:** Distributed caches often use replication to improve fault tolerance. When data is cached on one node, it is also copied to a backup node. the primary node becomes unavailable, the backup node can continue to serve data, ensuring uninterrupted access.

The diagram below shows the concept of consistent hashing to distribute keys acro nodes.



How Data Retrieval Works in Distributed Caching

Here's a step-by-step example to demonstrate how data retrieval works in a distribu cache, highlighting the process of a cache hit versus a cache miss:

- **Data Request**: Suppose an application needs to retrieve a frequently accessed piece of data, such as a user's session details. The application sends a request fo this data to the cache.
- **Determining Data Location**: The client library receives the request and, using consistent hashing, determines the appropriate cache node that should contain this specific piece of data based on its key.
- **Cache Hit Scenario**:
  - If the data is found on the designated cache node (a "cache hit"), the cache node retrieves the data and sends it back to the client library.
  - The client library then returns the data to the application, fulfilling the request quickly without the need to query the primary database.
  - Cache hits reduce latency and improve application performance by deliveri the requested data directly from the cache.
- **Cache Miss Scenario**:
  - If the requested data is not available on the cache node (a "cache miss"), the client library forwards the request to the primary database or data source.
  - Once the database retrieves the data, it is returned to the application. The client library may then store this data on the appropriate cache node so it's available for future requests, converting the cache miss into a cache hit on subsequent requests.
- **Data Replication** (**Optional**): If replication is enabled, the cached data may also copied to a backup node to ensure high availability. Should the primary node fa the client library can route future requests to the backup node.
- **Load Balancing via Sharding**: Sharding ensures that large data sets and heavy request loads are evenly distributed across cache nodes. When a new cache nod

added, consistent hashing helps rebalance the data distribution without significant data reallocation.

Hosting and Deployment Options

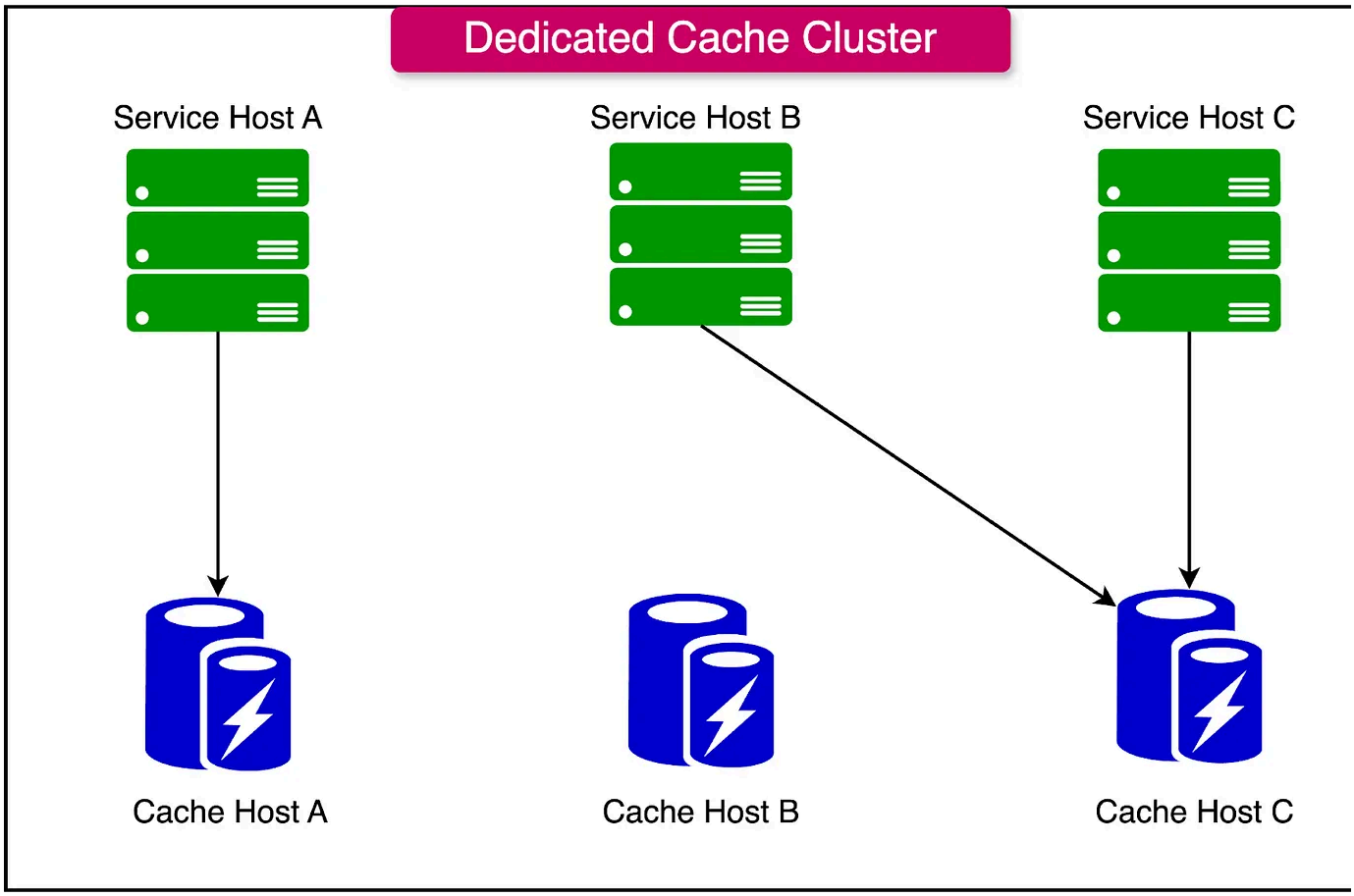
When implementing distributed caching, organizations have various hosting optio to choose from, each with its benefits and trade-offs.

Let us compare the main options for hosting distributed caches: dedicated cache servers, co-located caching, and cloud-based caching services.

Dedicated Cache Nodes

Dedicated cache nodes are standalone machines or virtual instances specifically allocated for caching.

See the diagram below:



They are separate from application servers, allowing them to handle caching tasks exclusively, without interference from application-level processes.

The main advantages of this setup are as follows:

- **Scalability**: Dedicated cache servers can be independently scaled to match application growth. As demand increases, more dedicated cache nodes can be added without affecting application servers, making it a flexible solution for larger, high-traffic applications.
- **Resource Isolation**: Since dedicated cache servers are separate from applicatio servers, they prevent cache-related activities from consuming resources needec for application processing. This isolation helps maintain stable performance, a caching processes don't compete for CPU or memory with other tasks.

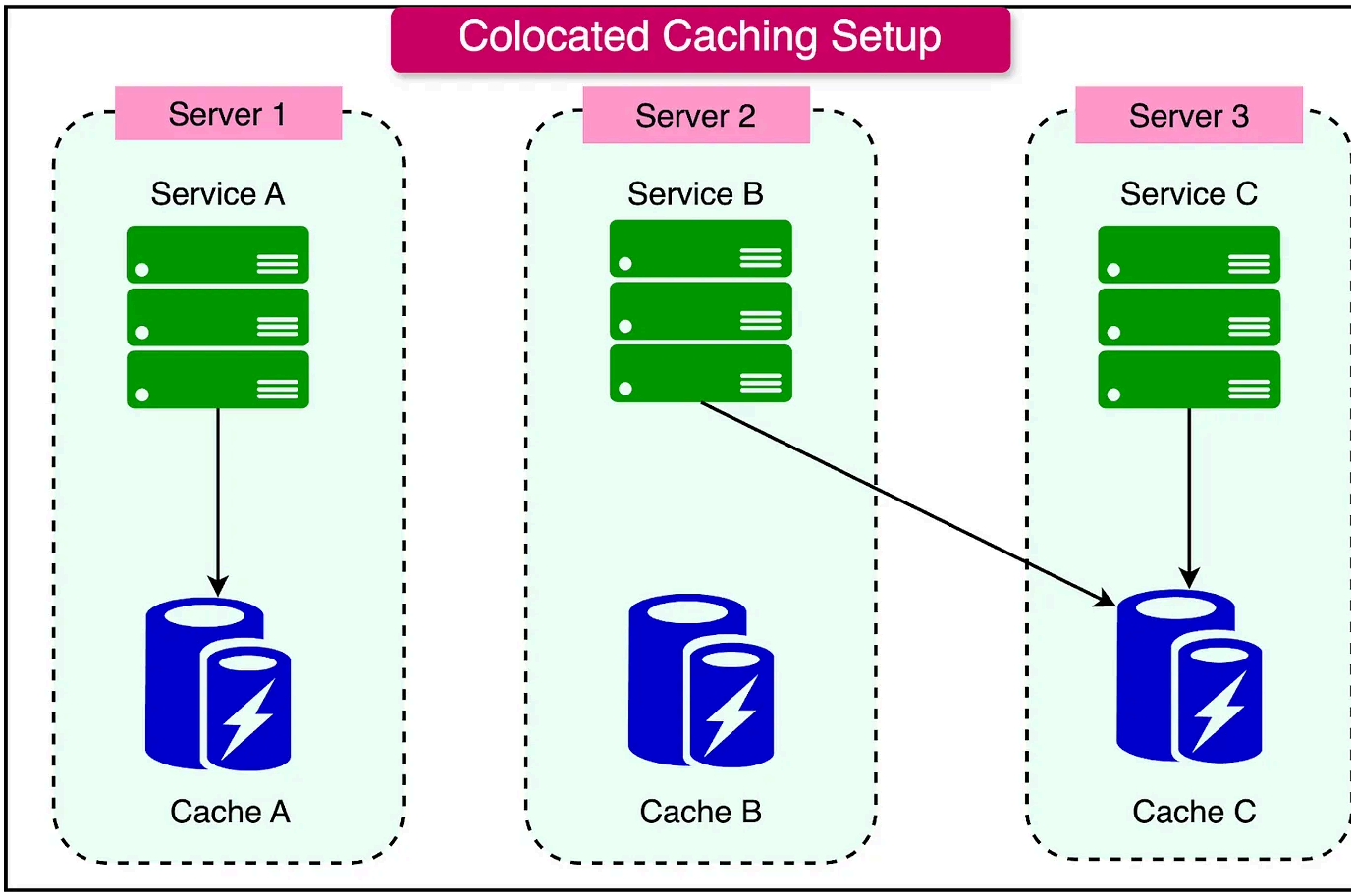
The disadvantages are as follows:

- **Network Latency**: Because dedicated cache servers are separate from applicati servers, accessing cached data requires network calls between the servers. This can introduce latency, especially in real-time applications where even minor delays impact performance.
- **Higher Costs**: Operating separate cache servers can be expensive, particularly when handling high cache traffic or when redundancy is required for fault tolerance. These costs can add up, as each dedicated server incurs hardware, maintenance, and energy expenses.

Co-located Caching

In co-located caching, the cache and application server run on the same physical machine or virtual instance.

See the diagram below for reference:



This setup leverages shared resources for both caching and application processes, which can simplify infrastructure management and reduce operational costs.

Here are the main advantages of this setup:

- **Low Latency**: Since the cache resides on the same server as the application, dat retrieval is faster and eliminates the network latency associated with remote ca servers.
- **Cost Efficiency**: Co-locating the cache with the application server is often mor cost-effective than dedicated caching, especially for smaller applications.

However, this setup also has some drawbacks:

- **Resource Contention**: With caching and application processes sharing the sam resources (CPU, memory, I/O), there is a risk of resource contention. High cach

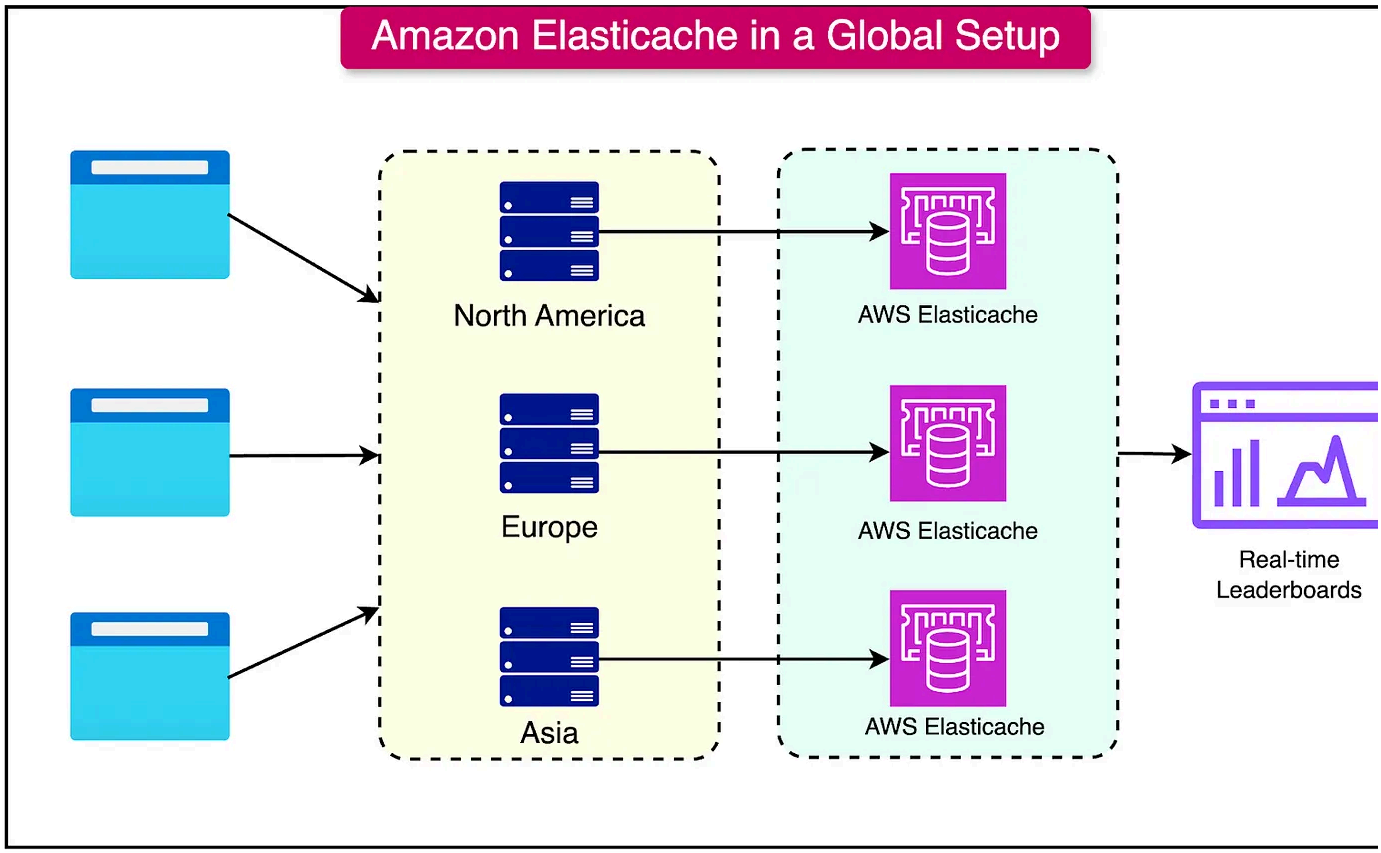
usage could strain the server, affecting application performance, especially und heavy load.

- **Limited Scalability**: Co-located caching setups are less scalable than dedicated cache servers. Scaling the cache would require upgrading the entire server, whi may be cost-prohibitive or technically challenging for large-scale applications.

Cloud-Based Caching Solutions (e.g., Amazon ElastiCache)

Cloud-based caching services, such as Amazon ElastiCache, offer fully managed, scalable caching solutions that can be set up in minutes and adapted to various application requirements.

See the diagram below for an example of such a setup on a global scale.



These solutions support popular caching engines, including Redis and Memcached and provide a suite of features tailored for enterprise-scale caching.

The key advantages of this setup are as follows:

- **Auto-Scaling**: Cloud caching solutions typically offer auto-scaling capabilities, allowing the cache infrastructure to automatically adjust based on demand. Th dynamic scaling is extremely useful during unexpected traffic surges.
- **Multi-Zone Deployment**: Managed cloud services support multi-AZ (Availabili Zone) deployments, ensuring high availability and fault tolerance.
- **Simplified Management**: Cloud providers handle setup, maintenance, patchin and monitoring, allowing development teams to focus on application-level concerns rather than infrastructure. They also offer integration with other clou services, making it easier to manage distributed cache in a cloud-native environment.
- **Flexible Cost Model**: Cloud caching services often have pay-as-you-go pricing, which can be more cost-effective for growing applications. Organizations can scale up or down based on demand, paying only for the resources they use, whi is an advantage over fixed-cost on-premise solutions.

The drawbacks are as follows:

- **Dependency on Cloud Provider**: Using a managed cloud service can lead to vendor lock-in, as it may be challenging to migrate cache data or configuration another provider. This dependency can become a consideration for companies with strict data governance policies or multi-cloud strategies.
- **Network Costs and Latency**: Although data retrieval is fast, accessing cloud-hosted caches still involves network overhead, which can incur additional data transfer costs and latency, particularly for applications with extremely high dat access demands.

Caching Strategies in Distributed Systems

Distributed caching relies on various strategies to optimize data retrieval and ensu consistency between the cache and data sources.

These caching strategies are designed to accommodate different data access patter and application requirements, such as data freshness, latency, and consistency.

Let's look at them in more detail.

Cache-Aside (Lazy Loading)

Cache-aside, also known as lazy loading, is a strategy where data is loaded into the cache only when it's explicitly requested by the application.

In this model, the cache does not preemptively store data. Instead, when an application requests a specific data item:

- If the data is found in the cache (a cache hit), it is returned immediately, provid fast access.
- If the data is not in the cache (a cache miss), the application retrieves it from th primary data source, such as a database, and then stores a copy in the cache for future requests.

This strategy minimizes the amount of cached data, as only frequently accessed iter are stored. Cache-aside is popular because of its simplicity and efficiency; it's particularly useful when applications don't need to cache all data upfront and when cached data changes infrequently.

The pros of this strategy are as follows:

- Simple and efficient, only caching data as needed.
- Reduces memory usage by caching only popular or recently accessed items.

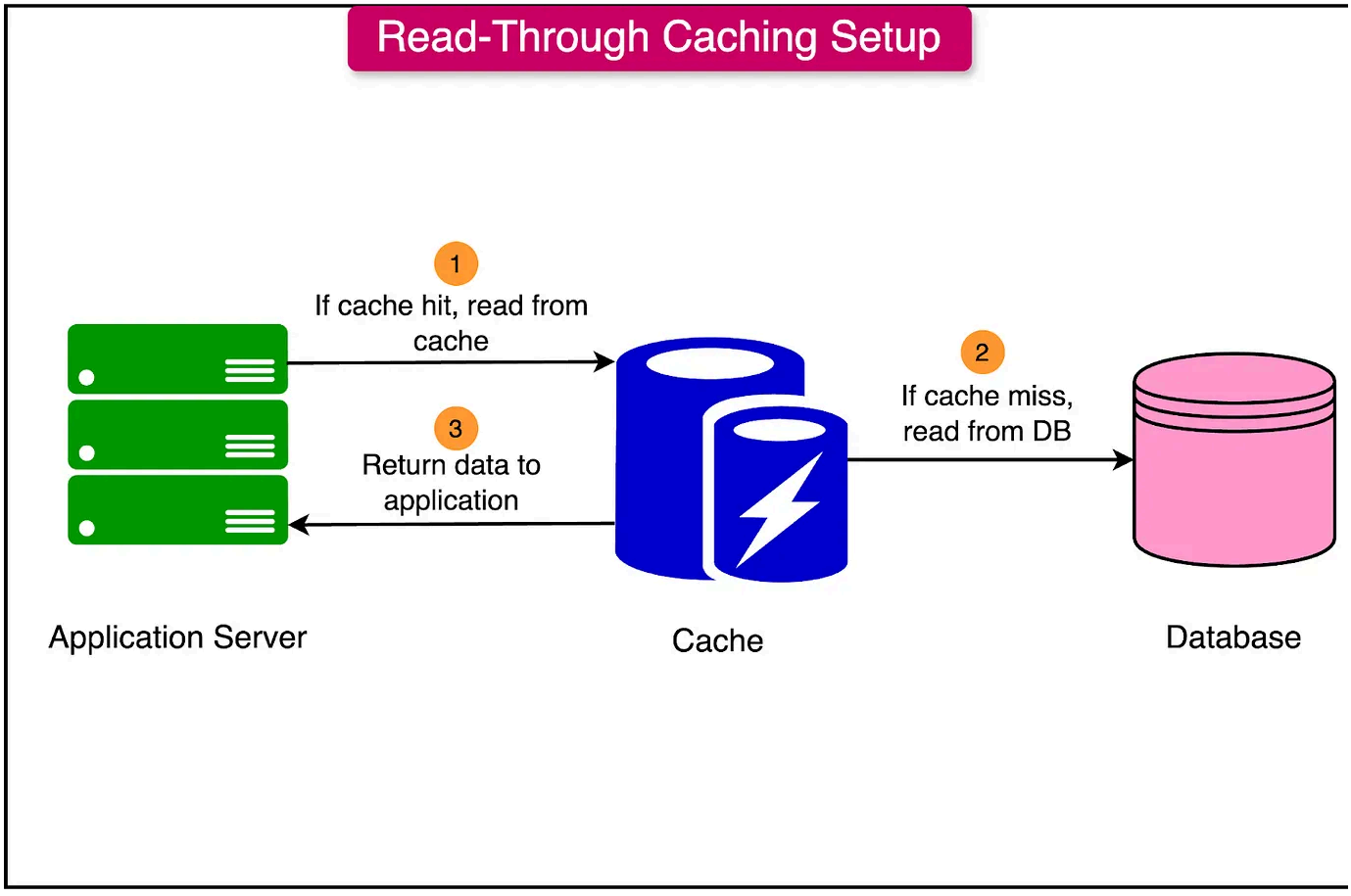
However, it also has a disadvantage:

- On a cache miss, there may be a delay as the data is retrieved from the database creating a "cold start" effect for less frequently accessed data.

Read-Through Caching

In a read-through caching strategy, the cache sits directly in front of the database, a all read requests pass through the cache. Here's how it works:

- When the application requests data, the cache checks if it has the requested ite
- If the item is present (cache hit), it is returned directly from the cache.
- If the item is not in the cache (cache miss), the cache itself retrieves the data fr the database, returns it to the application, and stores it in the cache.



Read-through caching is commonly used in managed caching solutions (e.g., Redis Amazon ElastiCache), as it keeps the caching mechanism within the cache layer, reducing the logic required in the application code.

The advantages of this approach are as follows:

- Streamlines the data flow by handling cache misses within the cache layer.
- Reduces cache management complexity in the application code.

There is also a disadvantage:

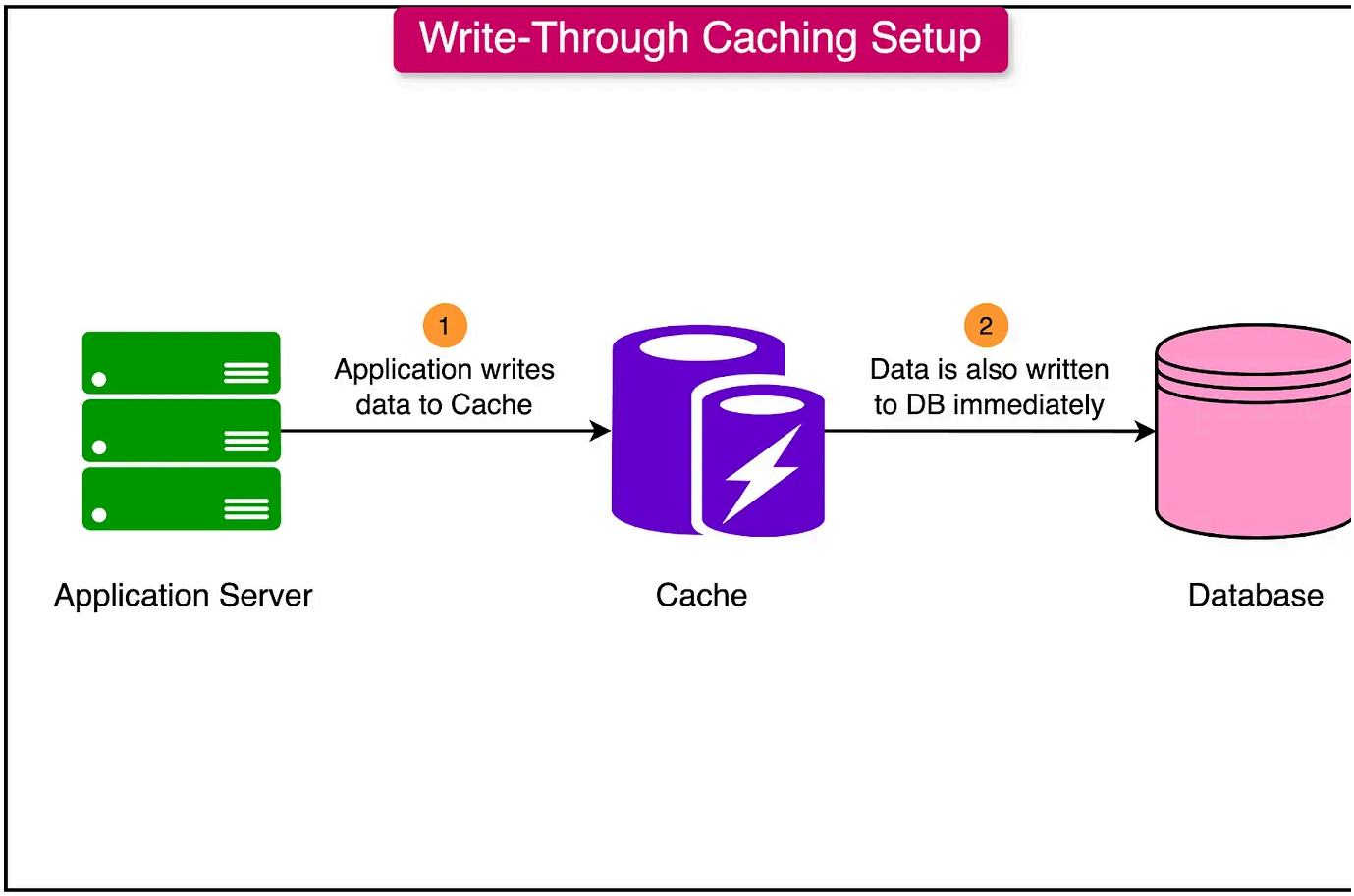
- Limited control over cache population, as all reads go through the cache.

Write-Through Caching

Write-through caching is a strategy that ensures data consistency by immediately writing updates to both the cache and the primary data store.

When an application updates data:

- The write operation is performed on the cache and then immediately forwarde the database.
- The updated value is stored in both the cache and the primary data source, keeping them consistent with each other.



Write-through caching is useful for applications that require data consistency, as it ensures that any changes to the data are immediately reflected in both the cache an the database. It's especially helpful in scenarios where read requests may come soo after a write operation, as the data in the cache is always up to date.

Here are the key advantages of this approach:

- Keeps data in the cache and the database consistent at all times.
- Ideal for applications with frequent updates where cached data must remain current.

However, there are also some disadvantages:

- Write operations can be slower, as they must update both the cache and the database.

- Increased complexity in handling large volumes of write operations.

Write-Back (Write-Behind) Caching

Write-back caching, also known as write-behind caching, optimizes write operatio by temporarily storing updates in the cache and deferring their write to the primary data store.

Here's how it works:

- When the application writes data to the cache, the update is held in the cache without immediately writing it to the database.
- The cache periodically synchronizes updates with the database in the backgrou which reduces the number of direct writes to the database and improves overal write performance.

The advantages of this approach are as follows:

- It enhances performance by reducing the number of immediate writes to the database.
- Suitable for write-intensive applications that can tolerate some latency in data persistence.

The main disadvantages are as follows:

- Increased risk of data loss in case of cache failure before writes are synchronize
- Potential for cache inconsistency, especially if reads occur before the cache wri back to the database.

Cache Consistency and Synchronization in Write-Heav Applications

Maintaining consistency between the cache and the primary data source is a critica challenge, especially in write-heavy applications where data is frequently updated.

Here are some strategies to ensure cache consistency:

- **Cache Invalidation:** This strategy removes stale or outdated data from the cach ensuring that the next request will pull fresh data from the database. Invalidati can be triggered based on time (Time-to-Live or TTL) or events (e.g., a database update).
- **Write-Through Consistency:** Write-through caching, as discussed, maintains consistency by updating both the cache and the database simultaneously. Whil this approach can introduce latency, it ensures that cached data is always synchronized with the primary data source.
- **Event-Based Updates:** For applications with complex data flows, an event-base approach can be used to synchronize cache data. For example, if a database upd

event occurs, a corresponding event can trigger the cache to update or invalida affected items, keeping the cache in sync.

- **Leasing:** In a distributed cache environment, multiple nodes may try to modify same data simultaneously. Leasing ensures that only one operation can update cache entry at a time, preventing conflicts and maintaining data consistency across nodes.

Challenges with Distributed Caching

Distributed caching does not come without its own set of challenges. A lot of the challenges associated with distributed systems show up when dealing with distribu caching.

Here are a few major challenges along with their possible mitigation strategies:

Data Consistency Issues

In distributed caching, data consistency becomes a significant challenge, especially when cached data requires frequent updates. In applications with high write frequencies, ensuring that all cache nodes hold the most recent data is complex.

If one node holds stale data while another has the updated value, it can lead to inconsistent application behavior and errors.

The mitigation strategies for this situation are as follows:

- **Write-Through Caching:** Updates the cache and the primary data store simultaneously, keeping data in sync.
- **Event-Based Triggers:** Using events to synchronize cache and database update by invalidating or updating relevant cache entries when data changes.
- **Leasing:** Ensures that only one node can modify a cache entry at a time, preventing conflicts in write-heavy environments.

Network Partitioning

Network partitioning, or temporary network failures, can disrupt communication between cache nodes, isolating portions of the cache from the rest of the system. Tl "split-brain" scenario can result in nodes operating independently with outdated or inconsistent data.

During a partition, nodes may independently handle read/write requests, leading to divergent cache states that are difficult to reconcile when the network is restored.

The mitigation strategies for this challenge are as follows:

- **Partition-Aware Architectures:** Designing the system to handle partial data access gracefully in case of partitioning.
- **Quorum-Based Consensus:** Ensuring that only a subset of nodes can operate during a partition to avoid conflicting data states, requiring consensus for updates.

Cache Invalidation and Expiration Strategies

Invalidation and expiration strategies are essential for keeping cached data fresh, especially in applications where data changes frequently.

TTL settings automatically expire data from the cache after a specified duration, ensuring that stale data doesn't persist indefinitely. However, setting appropriate TTLs can be complex, as too short a TTL may cause excessive cache misses, while t long a TTL risks serving outdated data.

The key challenges are as follows:

- **Overhead of Frequent Invalidations:** Frequent invalidations, especially in high dynamic systems, can degrade cache performance and increase the load on the primary data store.

- **Complexity of Implementing Event-Based Invalidation:** Configuring invalidat to trigger accurately based on specific events (e.g., database updates) adds complexity to the cache management process.

For complex applications, manual or event-triggered invalidation can ensure that cache entries are cleared or updated precisely when changes occur.

Load Balancing and Latency

In distributed caching, balancing the load across multiple cache nodes is essential t prevent overloading any single node.

Replicating data across nodes, especially in real-time applications, can introduce latency. Data synchronization between nodes needs to be fast to maintain data freshness and prevent lag during high-load periods.

Data replication, especially in write-heavy systems, can slow down performance as nodes continually update the replicas.

Some mitigation strategies for this situation are as follows:

- **Consistent Hashing:** Ensures even data distribution and minimizes reallocatio when nodes are added or removed, improving load balancing.
- **Replication with Selective Sharding:** Dividing data into shards and only replicating high-priority data can reduce replication-related latency and resour consumption.
- **Monitoring and Adaptive Scaling:** Monitoring system load in real-time and dynamically adjusting cache node allocation can help balance traffic and minin latency.

Popular Distributed Caching Solutions

Let's now look at some popular distributed caching solutions available to developer Each of these caching solutions offers unique strengths and capabilities.

Redis

Redis is a widely used, formerly open-source, in-memory data store that supports various data structures, making it highly versatile as a cache, database, and message broker.

It has several strengths such as:

- **In-Memory Storage:** Stores data entirely in memory, making it extremely fast f read and write operations.
- **Eviction Policies:** Offers flexible eviction policies, including Least Recently Us (LRU) and Least Frequently Used (LFU), helping optimize memory usage.
- **Data Persistence:** Redis provides optional persistence through snapshotting an append-only file (AOF) modes, ensuring data durability even in case of power l
- **Advanced Data Structures:** Redis supports lists, sets, sorted sets, hashes, and more, making it ideal for complex caching needs.

The primary use cases for Redis are:

- Real-time analytics
- Session storage
- Leaderboards and counters
- Publish/subscribe message queues

Memcached

Memcached is a lightweight, open-source, in-memory caching system designed for simplicity and speed.

It's well-suited for applications that don't require persistent storage or complex dat structures.

The key strengths of Memcached are as follows:

- **Simplicity:** Memcached is straightforward to set up and use, focusing purely or caching without additional features.
- **High-Speed Performance:** With minimal overhead, Memcached is highly effici for quick data retrieval in applications with high read and write throughput.
- **Distributed Architecture:** Supports horizontal scaling, allowing data to be spre across multiple nodes.

The primary use cases are as follows:

- Database query caching
- Simple session storage
- Caching of rendered web pages or HTML fragments
- Reducing load on backend services in high-traffic web applications

Amazon ElastiCache

Amazon ElastiCache is a fully managed, scalable caching service from AWS that supports both Redis and Memcached, providing flexible deployment options for distributed caching.

Its fundamental strengths are as follows:

- **Managed Service:** AWS handles maintenance, patching, scaling, and failover, allowing developers to focus on application logic.
- **Multi-Zone Deployments:** Supports multi-zone deployments for high availabil and fault tolerance, making it suitable for mission-critical applications.

- **Auto-Scaling and Monitoring:** ElastiCache offers automated scaling and integrates with Amazon CloudWatch for real-time monitoring.
- **Choice of Engines:** Users can select between Redis for advanced data features and Memcached for lightweight caching needs.

The key use cases of ElastiCache are:

- Large-scale web applications
- E-commerce platforms with dynamic content caching
- Real-time data processing and high-performance applications

The table below summarizes the strengths and limitations of the three major distributed caching solutions we've discussed.

Comparison Between Distributed Caching Solutions

Solution	Strengths	Limitations	Use Cases
Redis	In-memory storage, flexible eviction policies, data persistence, supports complex data types	More resource-intensive, requires tuning for high write loads	Real-time analytics, session storage, leaderboards
Memcached	Simple setup, high-speed performance, suitable for non-persistent needs	Limited data structure support	Query caching, session storage, page caching
AWS ElastiCache	Managed service with support for Redis and Memcached, multi-zone deployments, auto-scaling	Cost considerations for high availability and scaling	Large-scale apps, real-time processing, e-commerce

Summary

In this article, we've taken a detailed look at distributed caching and its role in building high-performance applications.

Let's summarize our learnings in brief:

- Distributed caching is a caching technique that spreads data across multiple servers to improve scalability, performance, and fault tolerance in large-scale applications.
- The core components of a distributed cache include cache nodes, client librarie data replication, and techniques like sharding and consistent hashing.
- Common hosting options for distributed caching are dedicated cache servers, c located caching, and cloud-based caching solutions, each with distinct advanta and drawbacks.
- Key caching strategies in distributed systems include cache-aside (lazy loading) read-through, write-through, and write-back caching, each suited to different c access needs.
- Distributed caching presents several challenges, including data consistency iss network partitioning, cache invalidation, and load-balancing complexities.
- Popular distributed caching solutions include Redis, which offers in-memory storage, flexible eviction policies, and data persistence; Memcached, known for simplicity and speed; and Amazon ElastiCache, a managed service providing support for both Redis and Memcached with multi-zone deployments and auto scaling capabilities.

327 Likes · 17 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Gorty, Murthy Dec 2, 2024

Hi, trying to understand this con in write-back (write-behind) caching

>>

Potential for cache inconsistency, especially if reads occur before the cache writes back to th database.

>>

Shouldn't the app also be 'reading' from the cache, thus having consistent data even if it is nc persisted to the db?

LIKE REPLY

2 replies

2 more comments...